

Visión Artificial

Tema 3. Apéndice

Plano proyectivo. Representación de puntos y rectas en coordenadas homogéneas. Homografías planares.

PEDRO ENRIQUE LÓPEZ DE TERUEL ALCOLEA
DEPARTAMENTO INGENIERÍA Y TECNOLOGÍA DE COMPUTADORES
FACULTAD DE INFORMÁTICA

Universidad de Murcia

Máster en Inteligencia Artificial

2025/2026





- ▶ Truco que se emplea en geometría proyectiva para manejar fácilmente la no linealidad de la división por Z (profundidad de la escena) de la que depende siempre la formación de una imagen 2D a partir de una escena 3D.
- ▶ Con este truco, la mayor parte de las técnicas que usaremos podrán seguir basadas simplemente en el **álgebra lineal**.
- ▶ En 2D, se reduce a lo siguiente:

$$(x, y) \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

*Coordenadas de imagen
homogéneas*

$$\begin{bmatrix} x \\ y \\ w \end{bmatrix} \Rightarrow (x/w, y/w)$$

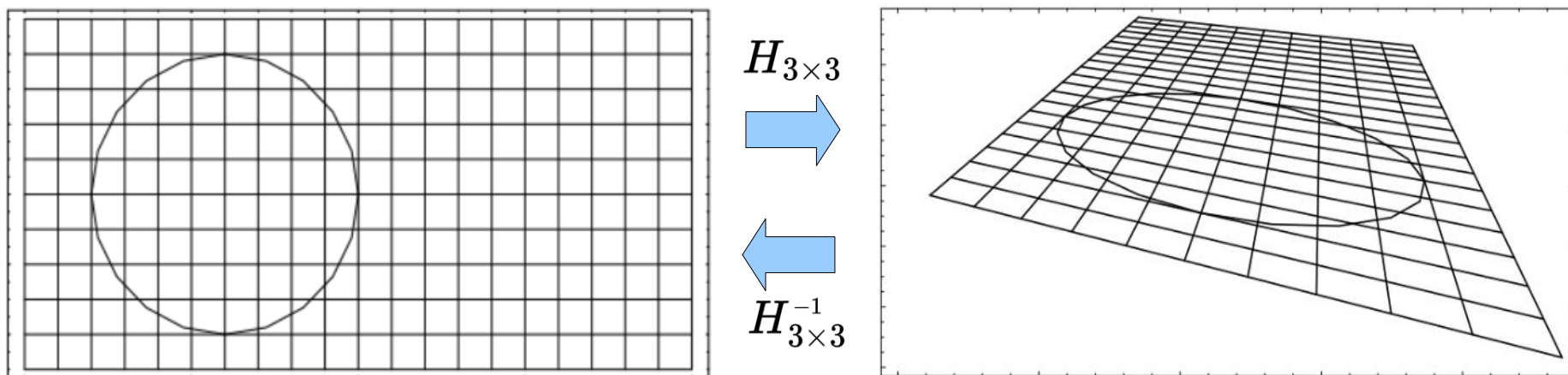
*Recuperación coordenadas 2D
desde coordenadas homogéneas*

Apéndice T3: Plano proyectivo

Transformaciones homográficas sobre imágenes



- La transformación de puntos 2D de un plano del mundo a un plano de imagen (y viceversa) se modela con una matriz H de tamaño 3×3 , denominada “homografía planar”.



- Una matriz $H_{3 \times 3}$ mapea, en general, puntos 2D a puntos 2D, trabajando sobre coordenadas homogéneas

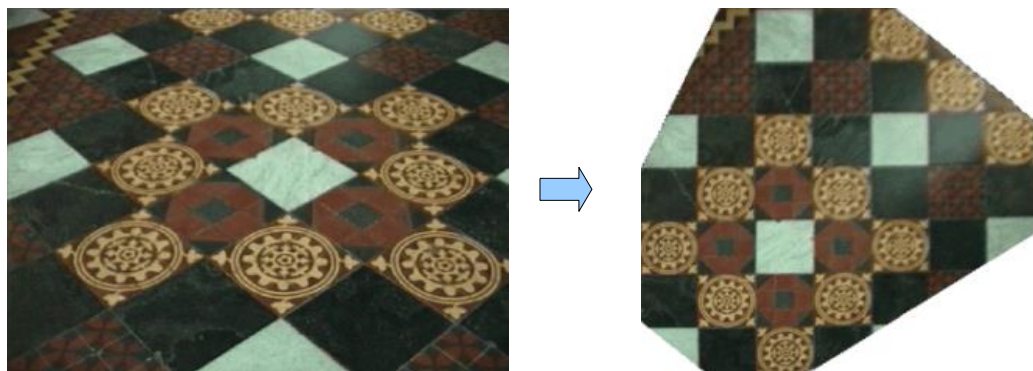
$$\begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \cdot \begin{bmatrix} x_{in} \\ y_{in} \\ w_{in} \end{bmatrix} = \begin{bmatrix} x_{out} \\ y_{out} \\ w_{out} \end{bmatrix}$$

Apéndice T3: Plano proyectivo

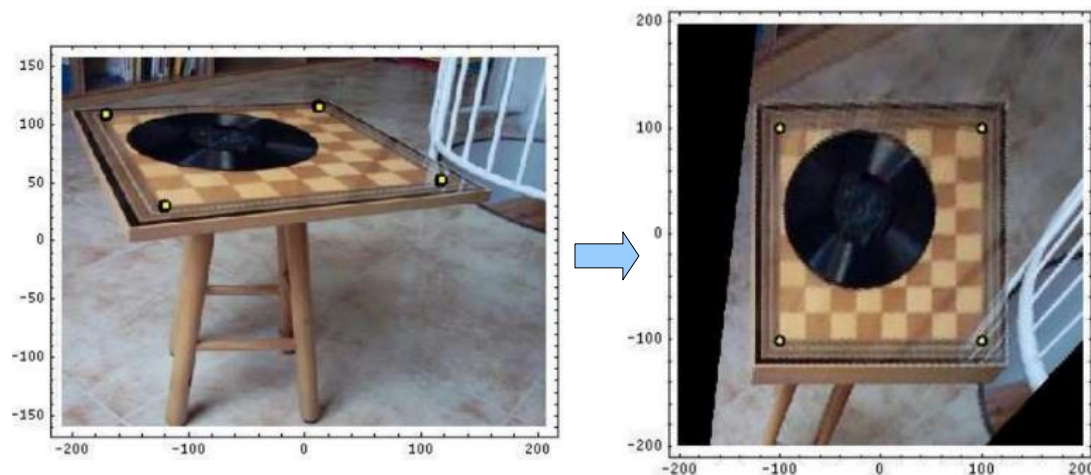
Ejemplo de aplicación



- Rectificación **frontoparalela** de un plano del mundo:



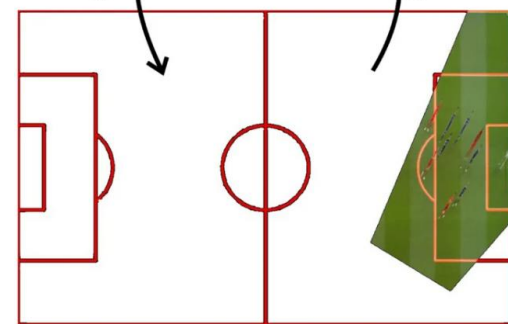
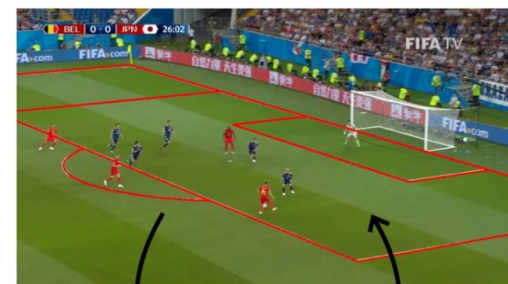
- Ejemplo de estimación de $H_{3 \times 3}$ a partir de puntos:
 - Se muestran las coordenadas originales y las deseadas (2D, aún no homogeneizadas), y la H correspondiente (H^{-1} llevaría de la imagen rectificada de nuevo a la original):



$$\begin{pmatrix} x & y \\ -171 & 109 \\ -120 & 31 \\ 117 & 53 \\ 11 & 115 \end{pmatrix} \rightarrow \begin{pmatrix} x' & y' \\ -100 & 100 \\ -100 & -100 \\ 100 & -100 \\ 100 & 100 \end{pmatrix}$$

$$H = \begin{bmatrix} 0.63868 & 0.77290 & -39.73059 \\ -0.11082 & 1.94177 & -165.90578 \\ -0.00034 & -0.00378 & 1 \end{bmatrix}$$

Aplicación en análisis (scouting) deportivo

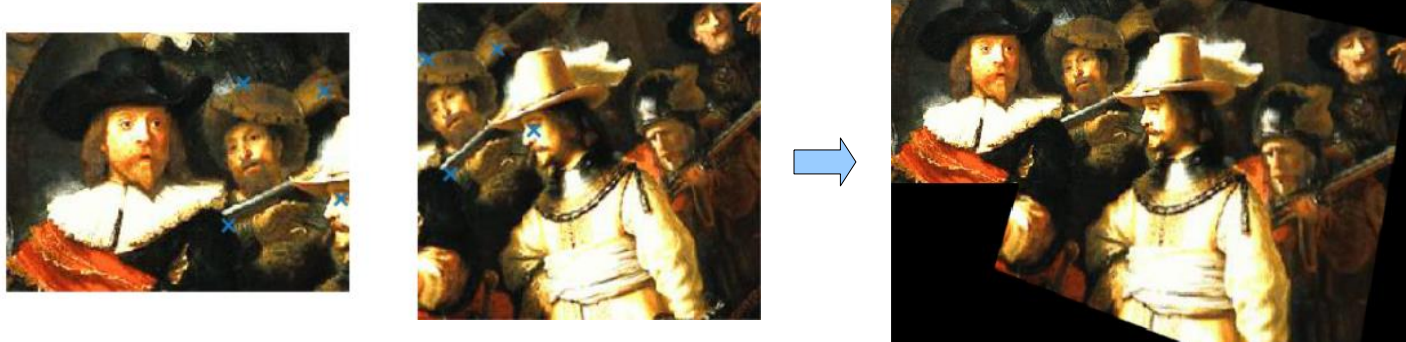
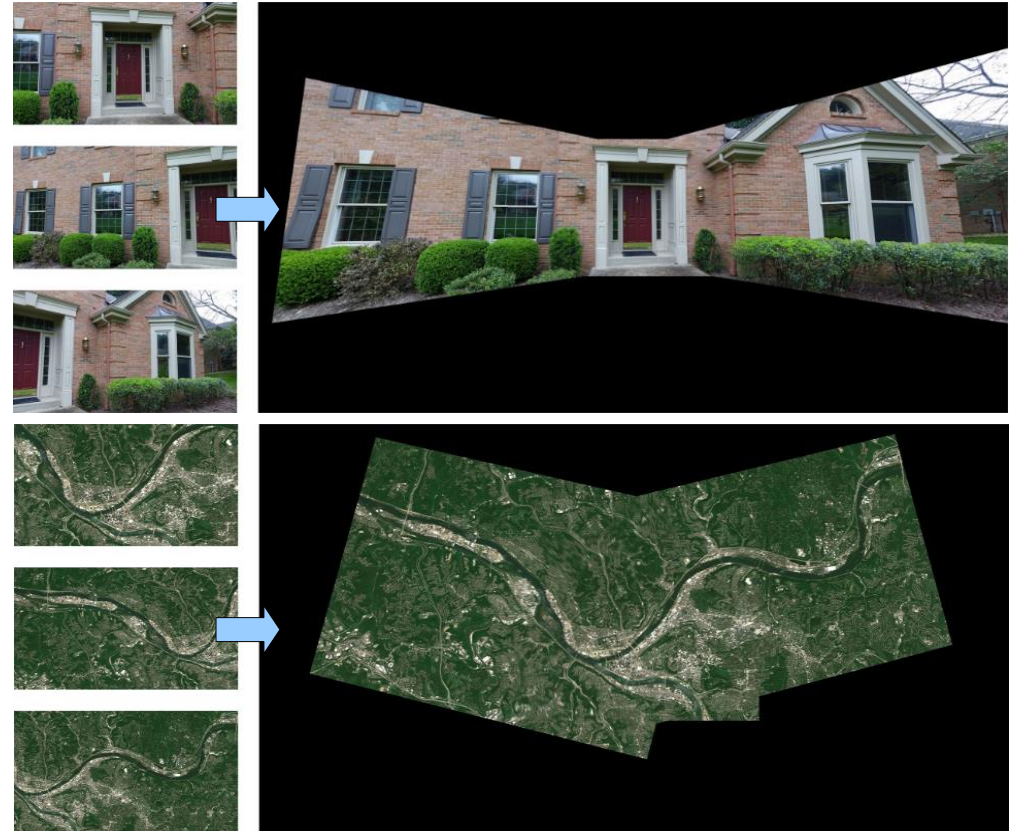


Apéndice T3: Plano proyectivo

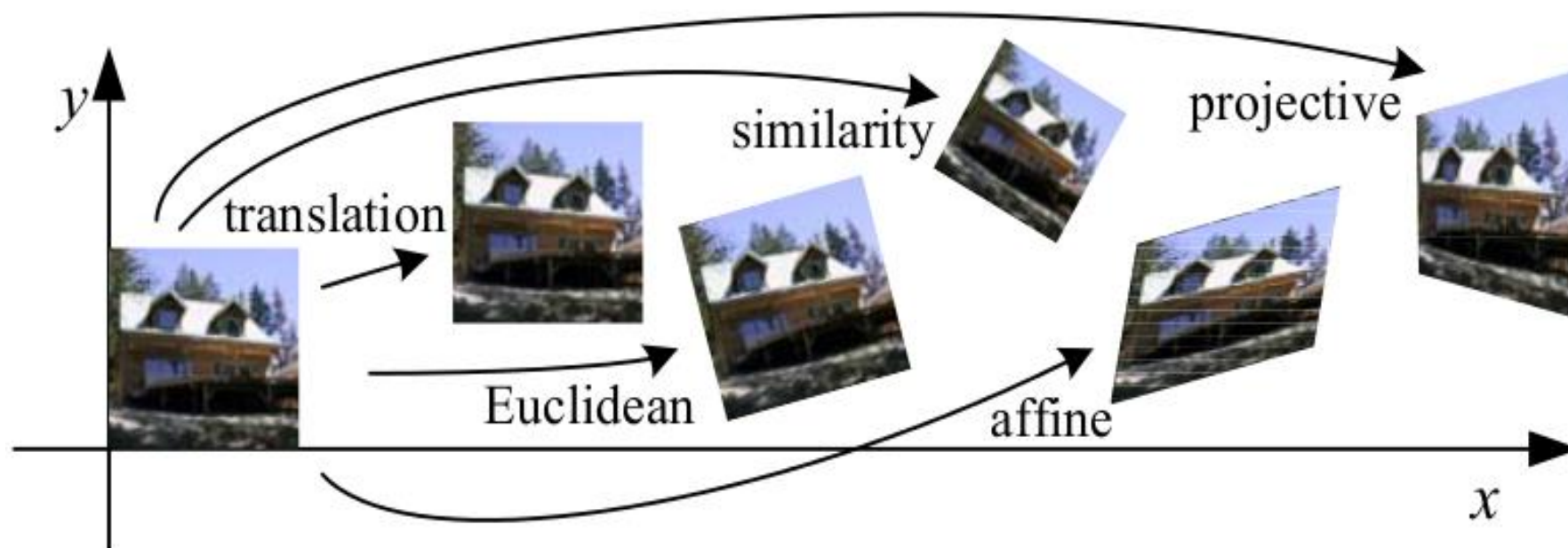
Otras aplicaciones: mosaicos de imágenes



- ▶ Varias vistas de una escena tomadas desde el mismo punto (pueden variar la orientación, zoom, etc.) se mueven a una referencia común usando homografías.
- ▶ Aplicado en *stitching* de imágenes satelitales (Google Maps, etc.)
- ▶ Usan correspondencias de (mínimo 4) puntos (tema 4):



► Repertorio básico de transformaciones geométricas (*warpings*) 2D:



Muy utilizadas para data augmentation, para evaluar corrección y precisión de matchings entre imágenes en condiciones controladas, para evaluar sensibilidad de modelos de detección/segmentación ante variaciones de perspectiva, etc.

Apéndice T3: Plano proyectivo

Transformaciones geométricas (II)



► Jerarquía de transformaciones 2D (funcionan con coordenadas homogéneas $(x,y,1)^T$):

Transformation	Matrix	# DoF	Preserves	Icon
translation	$\begin{bmatrix} \mathbf{I} & \mathbf{t} \end{bmatrix}_{2 \times 3}$	2	orientation	
rigid (Euclidean)	$\begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix}_{2 \times 3}$	3	lengths	
similarity	$\begin{bmatrix} s\mathbf{R} & \mathbf{t} \end{bmatrix}_{2 \times 3}$	4	angles	
affine	$\begin{bmatrix} \mathbf{A} \end{bmatrix}_{2 \times 3}$	6	parallelism	
projective	$\begin{bmatrix} \tilde{\mathbf{H}} \end{bmatrix}_{3 \times 3}$	8	straight lines	

Nota: Nosotros siempre añadiremos una tercera fila $(0,0,1)$ a las matrices 2×3 de esta tabla, para obtener siempre matrices $H_{3 \times 3}$ sin tener que tratar casos especiales

► Ejemplo de parametrización de matrices de transformación del plano 2D:

$$\begin{pmatrix} s_x \cos(\alpha) & -s_x \sin(\alpha) & t_x \\ s_y \sin(\alpha) & s_y \cos(\alpha) & t_y \\ 0 & 0 & 1 \end{pmatrix}$$

Rotación, traslación y
escalado anisotrópico
(euclídea)

$$\begin{pmatrix} 1 & a & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Skew
(afín)

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ p_1 & p_2 & 1 \end{pmatrix}$$

Perspectiva
(proyectiva)

- α es el ángulo de rotación,
- (t_x, t_y) es la traslación en los ejes X e Y,
- (s_x, s_y) son los escalados en dichos ejes,
- a es el *skew* (sesgo), y
- (p_1, p_2) son los parámetros que consiguen el efecto de perspectiva.

Apéndice T3: Plano proyectivo

Transformaciones geométricas (IV)



- ▶ Las matrices anteriores trabajan de forma muy intuitiva para coordenadas de puntos “normalizadas” (en el rango aproximado $[-1.0, +1.0]$; es decir, no directamente coordenadas de píxeles).
- ▶ En esas circunstancias, p.e., los rangos siguientes de parámetros recorren un grupo muy ilustrativo de transformaciones:

- $\alpha \in [-\pi, \pi]$
- $t_x \in [-5.0, +5.0]$
- $t_y \in [-3.5, +3.5]$

Rotación y traslación

- $s_x \in [0.0, 4.0]$
- $s_y \in [0.0, 4.0]$

Escalado

- $a \in [-1.0, +1.0]$
- $p_1 \in [-1.0, +1.0]$
- $p_2 \in [-1.0, +1.0]$

Afinidad y perspectiva

Apéndice T3: Plano proyectivo

Transformaciones geométricas (V)



- Las coordenadas de píxel NO están en el rango aproximado $[-1.0, +1.0]$, así que usaremos una matriz K que mapee coordenadas normalizadas a coordenadas de píxel (y por tanto K^{-1} hará lo contrario) para compensar:

$$\begin{pmatrix} s & 0 & w/2 \\ 0 & s & h/2 \\ 0 & 0 & 1 \end{pmatrix}, \text{ con: } \begin{cases} h = \text{altura de la imagen} \\ w = \text{anchura de la imagen} \\ s = \text{factor de escalado} \\ (\sim \text{mismo orden que } h/2 \text{ ó } w/2) \end{cases}$$

- Una transformación completa para, p.e., contornos de imagen, dados en coordenadas de píxel, quedaría:

$$H = K \cdot S \cdot A \cdot P \cdot K^{-1}, \text{ donde:}$$

$$(K, S, A, P) = \left(\begin{pmatrix} s & 0 & w/2 \\ 0 & s & h/2 \\ 0 & 0 & 1 \end{pmatrix}, \begin{pmatrix} s_x \cos(\alpha) & -s_x \sin(\alpha) & t_x \\ s_y \sin(\alpha) & s_y \cos(\alpha) & t_y \\ 0 & 0 & 1 \end{pmatrix}, \begin{pmatrix} 1 & a & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ p_1 & p_2 & 1 \end{pmatrix} \right)$$

Apéndice T3: Plano proyectivo

Tratamiento de rectas en imagen (I)

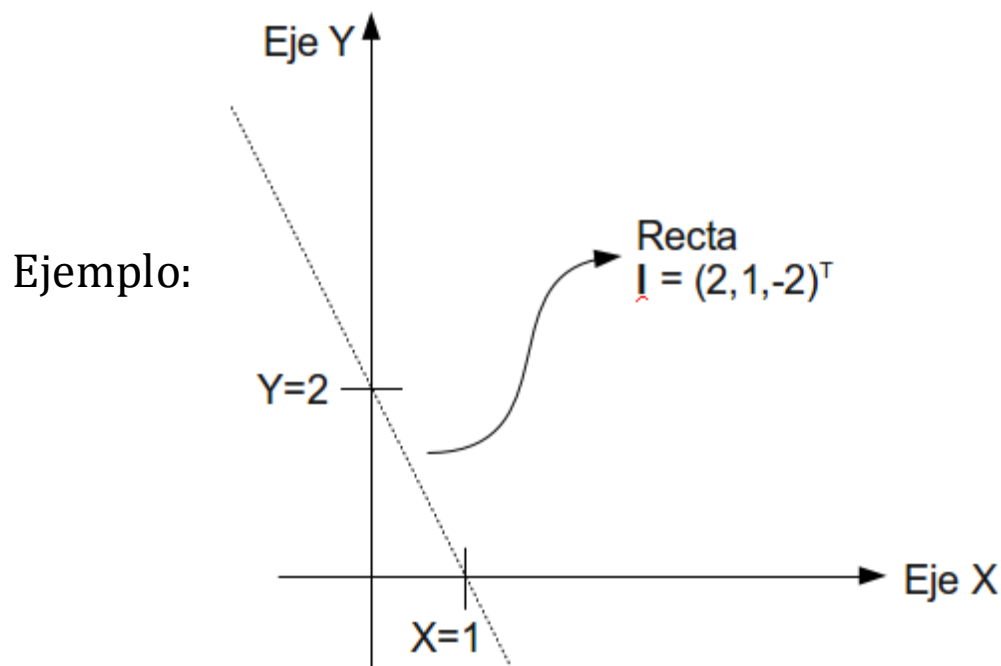


- Al igual que los puntos 2D, las rectas 2D serán tratadas como vectores homogéneos:

Coordenadas homogéneas de la recta

$$\mathbf{l} = \begin{bmatrix} a \\ b \\ c \end{bmatrix} \longrightarrow \mathbf{l}^T \cdot \mathbf{x} = 0 \Leftrightarrow ax + by + c = 0$$

Ecuación que cumplen los puntos $\mathbf{x} = (x, y, 1)$ de la recta

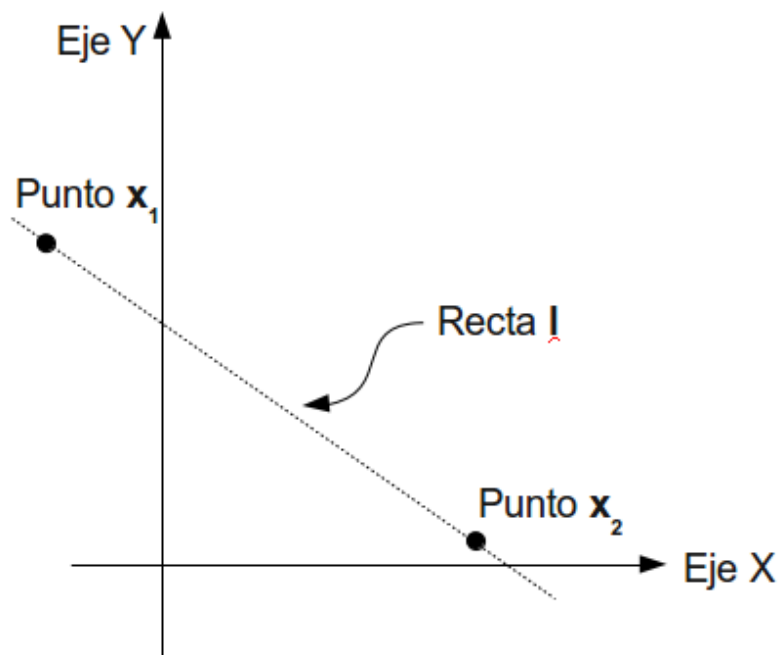
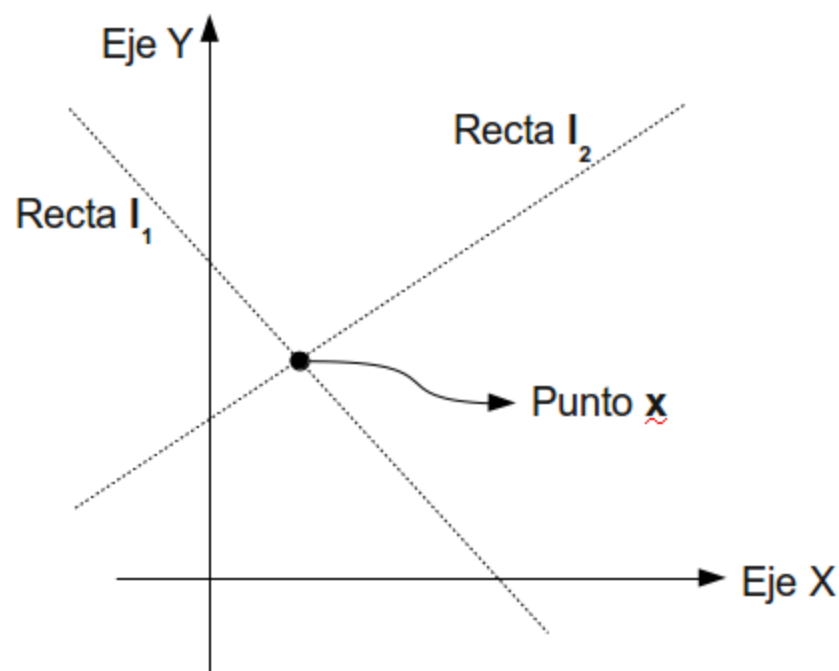


Apéndice T3: Plano proyectivo

Tratamiento de rectas en imagen (II)



- Las propiedades más interesantes de esta representación son la facilidad para determinar:
 - El punto $\mathbf{x}$ de intersección de dos rectas $\mathbf{l}_1$ y $\mathbf{l}_2$: $\mathbf{x} = \mathbf{l}_1 \times \mathbf{l}_2$
 - La recta $\mathbf{l}$ que pasa por dos puntos $\mathbf{x}_1$ y $\mathbf{x}_2$: $\mathbf{l} = \mathbf{x}_1 \times \mathbf{x}_2$



Apéndice T3: Plano proyectivo

Tratamiento de rectas en imagen (III)



- Recordar que el producto vectorial se computa así:

$$\mathbf{u} \times \mathbf{v} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ u_x & u_y & u_z \\ v_x & v_y & v_z \end{vmatrix} = \begin{vmatrix} u_y & u_z \\ v_y & v_z \end{vmatrix} \cdot \mathbf{i} - \begin{vmatrix} u_x & u_z \\ v_x & v_z \end{vmatrix} \cdot \mathbf{j} + \begin{vmatrix} u_x & u_y \\ v_x & v_y \end{vmatrix} \cdot \mathbf{k}$$

- es decir,

$$\mathbf{u} \times \mathbf{v} = \begin{bmatrix} u_x \\ u_y \\ u_z \end{bmatrix} \times \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix} = \begin{bmatrix} u_y v_z - u_z v_y \\ u_z v_x - u_x v_z \\ u_x v_y - u_y v_x \end{bmatrix}$$

Función
numpy
`np.cross()`

- Aunque a menudo se expresa simplemente como un producto matriz vector así:

$$\mathbf{a} \times \mathbf{b} = [\mathbf{a}]_{\times} \mathbf{b} = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix} \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Notación muy
empleada:
matriz “*cross product*”



- ▶ Otra ventaja de esta representación es que no se necesita ningún tratamiento especial para los **puntos y las líneas en el infinito**:
 - Los puntos en el infinito tienen la forma $\mathbf{x} = (x, y, 0)$.
 - La línea del infinito tiene la forma $\mathbf{l} = (0, 0, 1)$.
 - Las líneas paralelas se cortan en un punto en el infinito.
 - La línea que pasa por un punto en una dirección dada es la que une dicho punto con el punto en el infinito que hay en dicha dirección.
 - Etc...



► Cualquier operación en el plano se puede realizar de forma sencilla razonando sobre coordenadas homogéneas:

– **Ejemplo:** determinar distancia de un punto (x,y) a una recta $\mathbf{l}$:

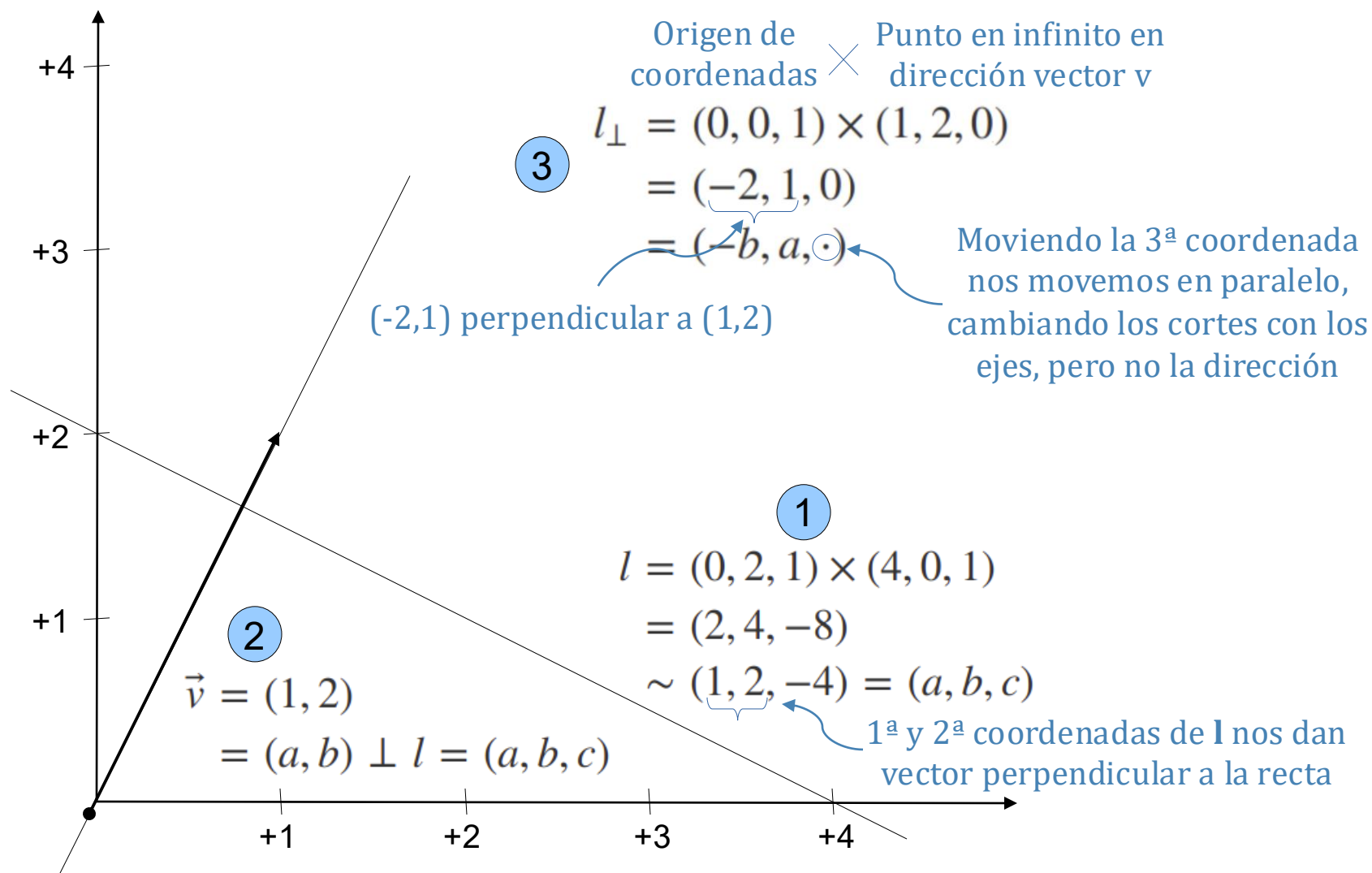
- Determinar punto de intersección (x^+,y^+) entre $\mathbf{l}$ y la línea $\mathbf{l}^\perp$ perpendicular a $\mathbf{l}$ que pasa por (x,y) .
- $D = \text{dist}((x^+,y^+),(x,y))$

Apéndice T3: Plano proyectivo

Tratamiento de rectas en imagen (VI)



► Rectas perpendiculares a l :

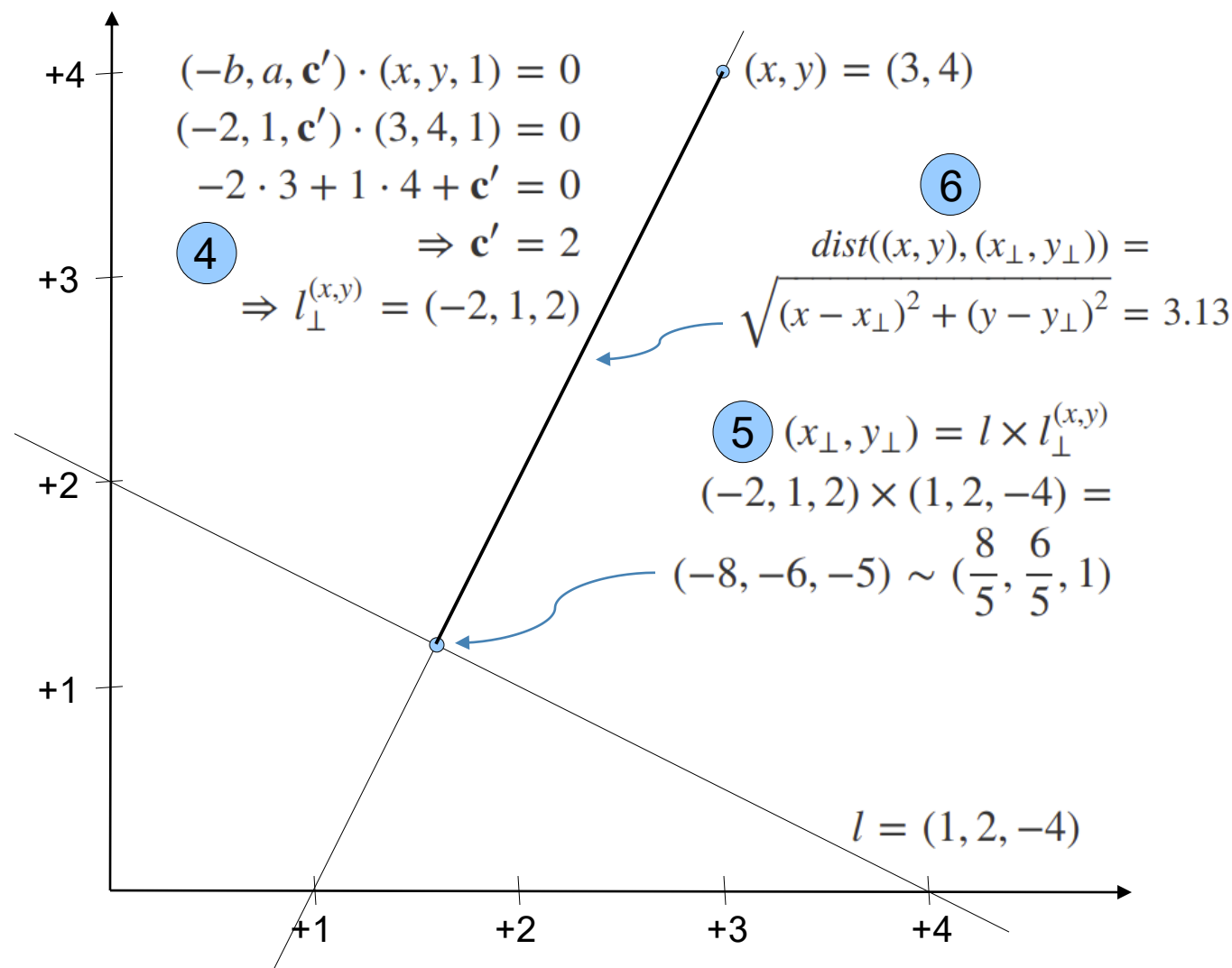


Apéndice T3: Plano proyectivo

Tratamiento de rectas en imagen (VII)



► Recta perpendicular que pasa por (x,y) , y distancia a l :



Apéndice T3: Plano proyectivo

Tratamiento de rectas en imagen (VIII)



- Finalmente, dada una homografía H que transfiere puntos $\mathbf{x} = (x, y, 1)$ según la ecuación:

$$\mathbf{x}' = \mathbf{H}\mathbf{x} \longleftrightarrow \begin{pmatrix} x'_1 \\ x'_2 \\ x'_3 \end{pmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$$

- Transfiere también líneas según la siguiente ecuación:

$$\mathbf{l}' = \mathbf{H}^{-\top} \mathbf{l}$$

- Es fácil razonar el porqué, algebraicamente:

$$\begin{aligned} l^\top x = 0 &\Leftrightarrow l^\top H^{-1} H x = 0 \Leftrightarrow l^\top H^{-1} x' = 0 \Leftrightarrow \\ l'^\top &= l^\top H^{-1} \Leftrightarrow l' = (l^\top H^{-1})^\top \Leftrightarrow l' = H^{-\top} l \end{aligned}$$



- ▶ Multiple view geometry (2nd ed.), R. Hartley, A. Zisserman (2003), Cambridge University Press.
- ▶ Tutorial homografías en OpenCV:
https://docs.opencv.org/4.x/d9/dab/tutorial_homography.html
- ▶ Apuntes de Visión Artificial, Alberto Ruiz (UMU):
<https://aruiz.org/vision/umucv/notebooks/coordhomog.html>